SDT Project Details

You can work in teams of three. Only one member of each team must take the quiz available on Moodle to receive the project number assigned to the team. When completing the quiz, that student must also provide the names of all team members.

Projects cannot be selected freely. Each team must implement the project assigned through Moodle.

For the backend implementation, you **must** use Spring Boot. For the frontend, you may use any additional technology of your choice.

The project will be presented in two stages and all team members must be present:

- **Exercises 1–6** will be presented in the middle of the semester.
- **Exercises 7–9** will be presented at the end of the semester.

If you present everything at the end, missing the first deliverable, some points will be deducted from the final project grade.

1. Event Ticketing and Reservation System

Develop an Event Ticketing and Reservation System for managing concerts, conferences, theatre performances, cinema screenings, or other events. Event organizers should be able to create events, configure ticket categories or seating areas, define prices, and manage ticket availability. Users should be able to browse events, reserve seats or tickets, complete simulated payments, cancel eligible reservations, and access their purchased tickets. The system should correctly manage concurrent reservation attempts and prevent the same seat or ticket from being assigned to multiple users. It should also support notifications related to purchases, cancellations, event changes, and reservation expiration.

Functionalities:

1. **Event Management**
    - Create, update, and remove events.
    - Configure event dates, locations, and capacity.
    - Define ticket categories and prices.
2. **Event Search**
    - Browse available events.
    - Search and filter events based on different criteria.
    - View event and ticket availability.
3. **Reservation Management**
    - Reserve tickets or seats.
    - Prevent duplicate reservations for the same seat.
    - Cancel or expire reservations.
4. **Ticket Purchase**
    - Simulate payment processing.
    - Generate tickets after successful purchases.
    - Allow users to view their purchased tickets.

5. **Notifications**
    - Notify users about successful purchases.
    - Send reservation expiration notifications.
    - Notify users when event information changes.

Project requirements:

1. (0.5p) Identify at least four design patterns that will be utilized in the project.
2. (0.5p) For each selected design pattern, provide a brief explanation justifying its use. This should detail how the pattern solves a specific problem and the advantages it offers over simpler alternatives.
3. (1 p) Design a single, comprehensive UML class diagram for the entire project. To highlight the pattern, you can use a note in UML, connected to all the classes involved in that pattern. Try to have a single design class diagram, do not treat the design patterns separately, instead they should be integrated in the entire app design.
4. (0.75p) Create two detailed UML sequence diagrams. These diagrams should illustrate the main use cases of the application.
5. (1.25p) Implement the design patterns discussed previously. Implementation in Java should be based on the design. The implementation will be limited in scope, focusing on selected functionalities to showcase design patterns. Focus on showcasing core functionalities rather than building the complete system. Limit the implementation to a proof-of-concept scope to demonstrate how the patterns interact with one another.
6. (2 p) Investigate, describe and evaluate three different software architectures for your system, each one using different architectural styles:
    - Monolithic architecture
    - Microservices architecture
    - Another distributed architecture style of your choice (e.g. peer-to-peer, event-driven etc.)

    For each architecture, you must:
    - Describe how the system would be structured in that style (main components, interactions, and data flow).
    - Provide a deployment and component to illustrate the architecture.
    - Discuss the pros and cons of using this style for your project.
    - Finally, compare the three alternatives and explain which one you consider the most suitable for your project, and why.
7. (2 p) Implement the project using a microservices architecture. The implementation should:
    - Contain at least two independent services, each with a clearly defined responsibility.
    - Demonstrate inter-service communication (e.g., via REST APIs, gRPC, or message queues).
    - Use Postman to test the microservices, provide in your submission a collection of postman requests covering all features.
    - Use Docker to package and run the services. Provide in Readme a clear instruction on how to run the system.

Focus on implementing core functionalities that showcase how the system works in a microservices style, rather than building every feature in full detail.

8. (1 p) Use a message queue (e.g., RabbitMQ, Kafka, or another suitable technology) for asynchronous communication between at least two services.
9. (1 p) Set up a simple continuous integration and deployment (CI/CD) pipeline (e.g., using GitHub Actions, GitLab CI, or Jenkins). The pipeline should automatically build, test, and deploy your services (at least to a local Docker environment).

2. Online Learning Platform

Create an Online Learning Platform where instructors can create courses and learning materials, while students can enroll in courses, complete assignments, take quizzes, and track their progress. The platform should also notify students about new materials, deadlines, and assessment results.

Functionalities:

1. **User and Course Management**
    - Register students and instructors.
    - Create and manage courses.
    - Allow students to enroll in available courses.
2. **Learning Content Management**
    - Add course materials and learning resources.
    - Organize content into modules or sections.
    - Publish and update learning materials.
3. **Assignment Management**
    - Create assignments with deadlines.
    - Allow students to submit assignments.
    - Track assignment submission status.
4. **Assessment and Progress Tracking**
    - Create quizzes or assessments.
    - Record assessment results.
    - Track student progress within courses.
5. **Notifications**
    - Notify students about new materials.
    - Send reminders about deadlines.
    - Notify users about course announcements and assessment updates.

Project requirements:

1. (0.5p) Identify at least four design patterns that will be utilized in the project.
2. (0.5p) For each selected design pattern, provide a brief explanation justifying its use. This should detail how the pattern solves a specific problem and the advantages it offers over simpler alternatives.
3. (1 p) Design a single, comprehensive UML class diagram for the entire project. To highlight the pattern, you can use a note in UML, connected to all the classes involved in that pattern. Try to have a single design class diagram, do not treat the design patterns separately, instead they should be integrated in the entire app design.
4. (0.75p) Create two detailed UML sequence diagrams. These diagrams should illustrate the main use cases of the application.
5. (1.25p) Implement the design patterns discussed previously. Implementation in Java should be based on the design. The implementation will be limited in scope, focusing on selected functionalities to showcase design patterns. Focus on showcasing core functionalities rather than building the complete system. Limit the implementation to a proof-of-concept scope to demonstrate how the patterns interact with one another.

6. (2 p) Investigate, describe and evaluate three different software architectures for your system, each one using different architectural styles:
    - Monolithic architecture
    - Microservices architecture
    - Another distributed architecture style of your choice (e.g. peer-to-peer, event-driven etc.)

    For each architecture, you must:
    - Describe how the system would be structured in that style (main components, interactions, and data flow).
    - Provide a deployment and component to illustrate the architecture.
    - Discuss the pros and cons of using this style for your project.
    - Finally, compare the three alternatives and explain which one you consider the most suitable for your project, and why.
7. (2 p) Implement the project using a microservices architecture. The implementation should:
    - Contain at least two independent services, each with a clearly defined responsibility.
    - Demonstrate inter-service communication (e.g., via REST APIs, gRPC, or message queues).
    - Use Postman to test the microservices, provide in your submission a collection of postman requests covering all features.
    - Use Docker to package and run the services. Provide in Readme a clear instruction on how to run the system.

Focus on implementing core functionalities that showcase how the system works in a microservices style, rather than building every feature in full detail.

8. (1 p) Use a message queue (e.g., RabbitMQ, Kafka, or another suitable technology) for asynchronous communication between at least two services.
9. (1 p) Set up a simple continuous integration and deployment (CI/CD) pipeline (e.g., using GitHub Actions, GitLab CI, or Jenkins). The pipeline should automatically build, test, and deploy your services (at least to a local Docker environment).

3. Smart Parking Management System

Create a Smart Parking Management System that allows drivers to find available parking spaces, reserve parking spots, enter and leave parking areas, and make payments. Parking administrators should be able to manage parking locations, prices, and availability.

Functionalities:

1. **Parking Location Management**
    - Create and manage parking locations.
    - Configure parking spaces and their properties.
    - Define pricing rules for each location.
2. **Parking Availability**
    - Display available and occupied parking spaces.
    - Update parking space status in real time.
3. **Reservation Management**
    - Allow users to reserve available parking spaces.
    - Cancel or expire reservations.
    - Prevent multiple reservations of the same parking space.
4. **Parking Sessions**
    - Start and end parking sessions.
    - Record entry and exit times.
    - Calculate parking costs based on duration.
5. **Payments and Notifications**
    - Simulate payment processing.
    - Notify users about reservations, expirations, and completed parking sessions.

Project requirements:

1. (0.5p) Identify at least four design patterns that will be utilized in the project.
2. (0.5p) For each selected design pattern, provide a brief explanation justifying its use. This should detail how the pattern solves a specific problem and the advantages it offers over simpler alternatives.
3. (1 p) Design a single, comprehensive UML class diagram for the entire project. To highlight the pattern, you can use a note in UML, connected to all the classes involved in that pattern. Try to have a single design class diagram, do not treat the design patterns separately, instead they should be integrated in the entire app design.
4. (0.75p) Create two detailed UML sequence diagrams. These diagrams should illustrate the main use cases of the application.
5. (1.25p) Implement the design patterns discussed previously. Implementation in Java should be based on the design. The implementation will be limited in scope, focusing on selected functionalities to showcase design patterns. Focus on showcasing core functionalities rather than building the complete system. Limit the implementation to a proof-of-concept scope to demonstrate how the patterns interact with one another.
6. (2 p) Investigate, describe and evaluate three different software architectures for your system, each one using different architectural styles:
    - Monolithic architecture
    - Microservices architecture

- Another distributed architecture style of your choice (e.g. peer-to-peer, event-driven etc.)

For each architecture, you must:

- Describe how the system would be structured in that style (main components, interactions, and data flow).
- Provide a deployment and component to illustrate the architecture.
- Discuss the pros and cons of using this style for your project.
- Finally, compare the three alternatives and explain which one you consider the most suitable for your project, and why.

7. (2 p) Implement the project using a microservices architecture. The implementation should:
   - Contain at least two independent services, each with a clearly defined responsibility.
   - Demonstrate inter-service communication (e.g., via REST APIs, gRPC, or message queues).
   - Use Postman to test the microservices, provide in your submission a collection of postman requests covering all features.
   - Use Docker to package and run the services. Provide in Readme a clear instruction on how to run the system.

Focus on implementing core functionalities that showcase how the system works in a microservices style, rather than building every feature in full detail.

8. (1 p) Use a message queue (e.g., RabbitMQ, Kafka, or another suitable technology) for asynchronous communication between at least two services.
9. (1 p) Set up a simple continuous integration and deployment (CI/CD) pipeline (e.g., using GitHub Actions, GitLab CI, or Jenkins). The pipeline should automatically build, test, and deploy your services (at least to a local Docker environment).

4. Digital Banking System

Build a simplified Digital Banking System where users can manage bank accounts, perform transfers, view transactions, and receive notifications. Administrators should be able to manage accounts and monitor transactions.

Functionalities:

1. **Customer Management**
    - Create and manage customer profiles.
    - Associate customers with one or more bank accounts.
    - Manage account access.
2. **Account Management**
    - Create and manage bank accounts.
    - View account balances.
    - View account details and status.
3. **Transaction Management**
    - Transfer funds between accounts.
    - Validate available balances before transactions.
    - Maintain transaction history.
4. **Transaction Status and Monitoring**
    - Track pending, successful, and rejected transactions.
    - Allow administrators to inspect transactions.
    - Maintain consistent account balances after transactions.
5. **Notifications**
    - Notify customers about completed transactions.
    - Notify users about failed transactions or important account activity.

Project requirements:

1. (0.5p) Identify at least four design patterns that will be utilized in the project.
2. (0.5p) For each selected design pattern, provide a brief explanation justifying its use. This should detail how the pattern solves a specific problem and the advantages it offers over simpler alternatives.
3. (1 p) Design a single, comprehensive UML class diagram for the entire project. To highlight the pattern, you can use a note in UML, connected to all the classes involved in that pattern. Try to have a single design class diagram, do not treat the design patterns separately, instead they should be integrated in the entire app design.
4. (0.75p) Create two detailed UML sequence diagrams. These diagrams should illustrate the main use cases of the application.
5. (1.25p) Implement the design patterns discussed previously. Implementation in Java should be based on the design. The implementation will be limited in scope, focusing on selected functionalities to showcase design patterns. Focus on showcasing core functionalities rather than building the complete system. Limit the implementation to a proof-of-concept scope to demonstrate how the patterns interact with one another.
6. (2 p) Investigate, describe and evaluate three different software architectures for your system, each one using different architectural styles:
    - Monolithic architecture

- Microservices architecture
- Another distributed architecture style of your choice (e.g. peer-to-peer, event-driven etc.)

For each architecture, you must:

- Describe how the system would be structured in that style (main components, interactions, and data flow).
- Provide a deployment and component to illustrate the architecture.
- Discuss the pros and cons of using this style for your project.
- Finally, compare the three alternatives and explain which one you consider the most suitable for your project, and why.

7. (2 p) Implement the project using a microservices architecture. The implementation should:
   - Contain at least two independent services, each with a clearly defined responsibility.
   - Demonstrate inter-service communication (e.g., via REST APIs, gRPC, or message queues).
   - Use Postman to test the microservices, provide in your submission a collection of postman requests covering all features.
   - Use Docker to package and run the services. Provide in Readme a clear instruction on how to run the system.

Focus on implementing core functionalities that showcase how the system works in a microservices style, rather than building every feature in full detail.

8. (1 p) Use a message queue (e.g., RabbitMQ, Kafka, or another suitable technology) for asynchronous communication between at least two services.
9. (1 p) Set up a simple continuous integration and deployment (CI/CD) pipeline (e.g., using GitHub Actions, GitLab CI, or Jenkins). The pipeline should automatically build, test, and deploy your services (at least to a local Docker environment).

5. Recruitment and Job Application Platform

Develop a Recruitment and Job Application Platform that connects companies with candidates. Companies should be able to publish job openings, manage applications, schedule interviews, and update candidate statuses. Candidates should create professional profiles, search for available positions, submit applications, and monitor their progress through the recruitment process. The platform should support multiple stages of recruitment and provide notifications when application statuses or interview details change.

Functionalities:

1. **User and Profile Management**
    - Register candidates and recruiters.
    - Create and update candidate profiles.
    - Manage company profiles.
2. **Job Management**
    - Create, update, and close job openings.
    - Define job requirements and application deadlines.
    - Search and filter available positions.
3. **Application Management**
    - Submit applications for open positions.
    - Track application status.
    - Allow recruiters to review and manage applicants.
4. **Interview Management**
    - Schedule interviews.
    - Update interview information.
    - Record interview status and feedback.
5. **Notifications**
    - Notify candidates about application status changes.
    - Send interview reminders.
    - Notify recruiters about new applications.

Project requirements:

1. (0.5p) Identify at least four design patterns that will be utilized in the project.
2. (0.5p) For each selected design pattern, provide a brief explanation justifying its use. This should detail how the pattern solves a specific problem and the advantages it offers over simpler alternatives.
3. (1 p) Design a single, comprehensive UML class diagram for the entire project. To highlight the pattern, you can use a note in UML, connected to all the classes involved in that pattern. Try to have a single design class diagram, do not treat the design patterns separately, instead they should be integrated in the entire app design.
4. (0.75p) Create two detailed UML sequence diagrams. These diagrams should illustrate the main use cases of the application.
5. (1.25p) Implement the design patterns discussed previously. Implementation in Java should be based on the design. The implementation will be limited in scope, focusing on selected functionalities to showcase design patterns. Focus on showcasing core

functionalities rather than building the complete system. Limit the implementation to a proof-of-concept scope to demonstrate how the patterns interact with one another.

6. (2 p) Investigate, describe and evaluate three different software architectures for your system, each one using different architectural styles:
    - Monolithic architecture
    - Microservices architecture
    - Another distributed architecture style of your choice (e.g. peer-to-peer, event-driven etc.)

    For each architecture, you must:
    - Describe how the system would be structured in that style (main components, interactions, and data flow).
    - Provide a deployment and component to illustrate the architecture.
    - Discuss the pros and cons of using this style for your project.
    - Finally, compare the three alternatives and explain which one you consider the most suitable for your project, and why.
7. (2 p) Implement the project using a microservices architecture. The implementation should:
    - Contain at least two independent services, each with a clearly defined responsibility.
    - Demonstrate inter-service communication (e.g., via REST APIs, gRPC, or message queues).
    - Use Postman to test the microservices, provide in your submission a collection of postman requests covering all features.
    - Use Docker to package and run the services. Provide in Readme a clear instruction on how to run the system.

Focus on implementing core functionalities that showcase how the system works in a microservices style, rather than building every feature in full detail.

8. (1 p) Use a message queue (e.g., RabbitMQ, Kafka, or another suitable technology) for asynchronous communication between at least two services.
9. (1 p) Set up a simple continuous integration and deployment (CI/CD) pipeline (e.g., using GitHub Actions, GitLab CI, or Jenkins). The pipeline should automatically build, test, and deploy your services (at least to a local Docker environment).

6. Hotel Management and Reservation System

Develop a Hotel Management and Reservation System for managing rooms, guests, reservations, and hotel services. Guests should be able to search for available rooms, make reservations, modify or cancel bookings, and perform simulated payments. Hotel staff should manage room availability, room types, prices, reservations, and guest information. The system should prevent conflicting reservations and support different pricing rules depending on room type, reservation period, or additional services.

Functionalities:

1. **Room Management**
    - Create and manage rooms and room types.
    - Define room prices and availability.
    - Track room status.
2. **Reservation Management**
    - Search for available rooms.
    - Create, modify, and cancel reservations.
    - Prevent overlapping reservations.
3. **Guest Management**
    - Register and manage guest information.
    - View previous and upcoming reservations.
    - Manage guest preferences.
4. **Check-in and Check-out**
    - Manage guest check-in.
    - Manage check-out and reservation completion.
    - Update room availability automatically.
5. **Payments and Notifications**
    - Simulate reservation payments.
    - Send booking confirmations.
    - Notify guests about reservation changes or upcoming stays.

Project requirements:

1. (0.5p) Identify at least four design patterns that will be utilized in the project.
2. (0.5p) For each selected design pattern, provide a brief explanation justifying its use. This should detail how the pattern solves a specific problem and the advantages it offers over simpler alternatives.
3. (1 p) Design a single, comprehensive UML class diagram for the entire project. To highlight the pattern, you can use a note in UML, connected to all the classes involved in that pattern. Try to have a single design class diagram, do not treat the design patterns separately, instead they should be integrated in the entire app design.
4. (0.75p) Create two detailed UML sequence diagrams. These diagrams should illustrate the main use cases of the application.
5. (1.25p) Implement the design patterns discussed previously. Implementation in Java should be based on the design. The implementation will be limited in scope, focusing on selected functionalities to showcase design patterns. Focus on showcasing core

functionalities rather than building the complete system. Limit the implementation to a proof-of-concept scope to demonstrate how the patterns interact with one another.

6. (2 p) Investigate, describe and evaluate three different software architectures for your system, each one using different architectural styles:
    - Monolithic architecture
    - Microservices architecture
    - Another distributed architecture style of your choice (e.g. peer-to-peer, event-driven etc.)

    For each architecture, you must:
    - Describe how the system would be structured in that style (main components, interactions, and data flow).
    - Provide a deployment and component to illustrate the architecture.
    - Discuss the pros and cons of using this style for your project.
    - Finally, compare the three alternatives and explain which one you consider the most suitable for your project, and why.
7. (2 p) Implement the project using a microservices architecture. The implementation should:
    - Contain at least two independent services, each with a clearly defined responsibility.
    - Demonstrate inter-service communication (e.g., via REST APIs, gRPC, or message queues).
    - Use Postman to test the microservices, provide in your submission a collection of postman requests covering all features.
    - Use Docker to package and run the services. Provide in Readme a clear instruction on how to run the system.

Focus on implementing core functionalities that showcase how the system works in a microservices style, rather than building every feature in full detail.

8. (1 p) Use a message queue (e.g., RabbitMQ, Kafka, or another suitable technology) for asynchronous communication between at least two services.
9. (1 p) Set up a simple continuous integration and deployment (CI/CD) pipeline (e.g., using GitHub Actions, GitLab CI, or Jenkins). The pipeline should automatically build, test, and deploy your services (at least to a local Docker environment).